TECHNICAL DEVELOPMENT LOG

Sparse Autoencoder Training for Vision Unlearning

LLaVA-NeXT · Llama3-8B · CLIP ViT-L/14-336

v2 (TopK-SAE) → v3 (L1-ReLU SAE) | April 2025

1. Overview & Motivation

This document logs the design rationale, architecture decisions, and iterative improvements made during the development of Sparse Autoencoder (SAE) training pipelines for **vision-based unlearning** in the LLaVA-NeXT Llama3-8B vision-language model (VLM). The goal is to learn interpretable, sparse representations of image patch activations inside the CLIP ViT-L/14-336 vision encoder — enabling selective suppression of specific visual features (e.g., a concept like **"zebra"**) without retraining the entire model.

Core Idea

Train a Sparse Autoencoder on internal vision encoder activations. Once trained, individual SAE latents correspond to interpretable visual features. By ablating (zeroing out) specific latents during inference, we can surgically 'unlearn' targeted visual concepts — a lightweight alternative to full model fine-tuning or RLHF-based unlearning.

1.1 Why SAEs for Vision Unlearning?

- **Interpretability:** SAEs decompose dense neural representations into sparse, monosemantic features — one latent ≈ one concept.
- **Surgical precision:** Ablating a single latent affects only its associated visual concept, leaving unrelated model behavior intact.
- **No retraining:** The SAE is trained post-hoc on frozen activations; the base VLM weights are never modified.
- **Scalability:** Once an SAE is trained for a layer, any number of concepts can be unlearned by identifying their corresponding latents.

1.2 Target Model Architecture

Field	Value
Full VLM	llava-hf/llama3-llava-next-8b-hf
Vision Encoder	CLIP ViT-L/14-336 (openai/clip-vit-large-patch14-336)
Hidden Dimension	d_model = 1024
Patches per Image	576 patch tokens (CLS dropped)
Key Layer (v2)	Layers 12, 17, 22 (0-indexed encoder layers)
Key Layer (v3/L1)	Layer 22 only (penultimate, closest to LLM)
Why Layer 22?	LLaVA reads from hidden_states[-2] — this is the richest semantic representation before the LLM projection layer

2. Version 2 — TopK Sparse Autoencoder (train_vision_sae_v2.py)

2.1 What We Built

v2 introduced a production-grade multi-layer SAE pipeline with TopK sparsity. This was the first version to correctly target the LLaVA-NeXT vision tower (v1 had incorrectly targeted LLaVA-1.5).

SAE Architecture — TopK

The TopK SAE enforces a hard sparsity budget by keeping only the **k largest activations** after the encoder ReLU, zeroing out the rest. This gives exact control over how many latents are active per token.

TopK SAE Formulas

Encoder: $h = \text{TopK}(\text{ReLU}(W_{enc} \cdot x_{norm} + b_{enc}),\ k=32)$ Decoder: $\hat{x}_{norm} = W_{dec} \cdot h + b_{dec}$ Loss: $\text{MSE} = \text{mean}(\|x_{norm} - \hat{x}_{norm}\|^2)$ Sparsity is enforced by TopK gate — exactly 32 latents active per token.

Key Design Decisions in v2

- **Tied init (W_enc = W_dec.T):** Initialising the encoder as the transpose of the decoder prevents dead latents from the start (from OpenAI scaling-SAE paper).
- **Preprocessing (mean-center + L2 norm):** Activations are centered by the dataset mean and divided by the mean L2 norm. This makes MSE loss comparable across layers and stabilises training.

- **Multi-layer training:** Separate SAEs trained for layers 12, 17, and 22. Layer 22 (penultimate) is the semantically richest and feeds directly into LLaVA's LLM projection.
- **Zebra oversampling:** The ImageNet training set includes up to 1,000 zebra images (class 340) alongside 49,000 random images to enrich concept-specific feature learning. This directly supports the unlearning use-case.
- **Dead feature tracking:** Features with zero activation across a sliding window of 50,000 tokens are flagged as 'dead'. High dead-feature counts indicate poor initialisation or over-sparsification.
- **L0 monitoring:** Reports the mean number of active features per token. With k=32 the theoretical L0 = 32; deviations signal issues with the TopK gate.
- **Explained Variance:** Reports $1 - \mathrm{Var}(x-\hat{x})/\mathrm{Var}(x)$ alongside MSE to assess reconstruction quality independently of scale.

2.2 Training Configuration

Parameter	Value
Expansion Factor	32× → d_sae = 32,768 latents
Top-K	k = 32 active latents per token
Dataset	ImageNet-1K (50,000 images; 1,000 zebra-oversampled)
Batch Size	4,096 activation vectors
Learning Rate	$3\times10^{-4} \rightarrow 1\times10^{-5}$ (cosine decay)
Epochs	5 passes over the full dataset
Log Interval	Every 50 SAE batches
Dead Feature Window	50,000 tokens
Checkpoint Frequency	End of every epoch

2.3 Training Modes

v2 supports two training modes:

- **Online mode (default):** Streams images directly from ImageNet, extracts activations on-the-fly with CLIP, and trains the SAE in a single pass. Avoids disk I/O overhead but requires a GPU throughout.
- **Offline mode (extract → stats → train):** Saves all activations to disk as chunk files first, then trains from disk. Useful for multi-run experiments on the same dataset.

2.4 Limitations of v2 (Why We Needed v3)

- **High L0 is fixed:** TopK forces exactly k=32 active features. This may be too many or too few depending on the input — it cannot adapt.

- **Large d_sae:** 32× expansion (32,768 latents) is very memory-intensive and slow to train.
- **No early stopping:** Training always runs for all 5 epochs, even if the SAE has converged.
- **Research alignment:** The L1 formulation (Pach et al. 2025) is more standard in the mechanistic interpretability literature, making results more comparable to published benchmarks.

3. Version 3 — L1-ReLU Sparse Autoencoder (train_vision_sae_l1.py)

3.1 What Changed and Why

v3 (internally labelled v3 in code) replaces TopK sparsity with **L1 regularisation**, aligning with the formulation in Pach et al. 2025 and standard interpretability SAE literature. It also narrows focus to **layer 22 only**, since prior experiments confirmed this layer holds the most semantically meaningful features.

SAE Architecture — L1-ReLU

Instead of a hard TopK gate, the encoder uses a plain ReLU. Sparsity is encouraged through an **L1 penalty term** in the loss function. The penalty coefficient λ (lambda) controls the trade-off between reconstruction fidelity and sparsity.

L1-ReLU SAE Formulas

Encoder: $h = \mathrm{ReLU}(W_{enc} \cdot (x - b_{dec}) + b_{enc})$ Decoder: $\hat{x} = W_{dec} \cdot h + b_{dec}$ Loss: $L = \mathrm{MSE}(x, \hat{x}) + \lambda \cdot \|h\|_1$ $\lambda = 0.03$ (l1_coeff). Sparsity emerges naturally from the penalty.

3.2 Architectural Differences Explained

Aspect	v2 (TopK)	v3 (L1-ReLU)
Sparsity method	TopK gate (hard, k=32)	L1 penalty, λ=0.03 (soft)
Encoder activation	TopK(ReLU(·))	ReLU(·) only
Expansion factor	32× → 32,768 latents	4× → 4,096 latents
Target layers	12, 17, 22	22 only

Training schedule	5 epochs, cosine LR decay	110,000 steps, cosine from step 109,999
Max images	50,000	20,000 (+ skip-if-exists)
Early stopping	Not implemented	MSE-based, warmup 10,000 steps
Activation caching	Always re-extracts	Skips if chunks already on disk
LR formula	Fixed 3×10^{-4}	16 / (125 × √d_sae) ≈ dynamic per d_sae

3.3 Key Improvements in v3

L1 Sparsity — Softer, More Adaptive

L1 penalisation lets sparsity **emerge naturally**: features that carry little information are penalised toward zero. Unlike TopK, different inputs can activate a different number of latents. This more closely mirrors how biological systems represent information and produces cleaner, more monosemantic features.

Smaller, Faster SAE (4× vs 32×)

Reducing the expansion factor from 32 to 4 cuts the SAE from 32,768 latents down to 4,096. Per the Pach et al. 2025 reference pipeline, a 4× expansion at layer 22 of CLIP ViT-L is sufficient to capture semantically meaningful features. Training is substantially faster and requires less GPU memory.

Early Stopping

v3 introduces a rolling MSE-based early stopping mechanism:

- **Warmup:** No stopping check for the first 10,000 steps (loss is still dropping steeply).
- **Patience:** If MSE does not improve by more than 1×10^{-6} for 5,000 consecutive steps, training halts.
- **State persistence:** The early stopper state is saved in checkpoints, enabling correct resumption from mid-training.

Skip-If-Exists for Activation Extraction

Phase 1 now checks whether activation chunk files already exist for a layer before running the full CLIP forward pass. This avoids 2–3 hours of redundant computation on repeated training runs — a significant quality-of-life improvement during hyperparameter sweeps.

Step-Based Schedule (aligned with reference shell script)

v3 adopts the Pach et al. reference hyperparameters from `monosemanticity_score.sh`: 110,000 total steps with cosine LR decay kicking in at step 109,999. The learning rate is computed as `16 / (125 × √d_sae)` — a formula that scales automatically with the SAE width.

4. Three-Phase Training Pipeline

Both versions share a three-phase pipeline structure. v3 adds skip logic in Phase 1.

Phase	What Happens	Duration (approx)	GPU Needed?
Phase 1: Extract	Run CLIP ViT-L forward passes on up to 20K ImageNet images. Save 576 patch activations per image per target layer as chunk .pt files.	2–3 hrs	Yes
Phase 2: Stats	Compute per-layer mean vector and mean L2 norm across all chunks. Used to normalise activations before SAE training.	~5 min	No (CPU OK)
Phase 3: Train	Load normalised activation chunks, train L1-ReLU SAE for up to 110,000 steps with early stopping and cosine LR decay.	1–2 hrs/layer	Yes

4.1 Data: Zebra Oversampling Strategy

The dataset is composed of up to **1,000 zebra images** (ImageNet class 340) mixed with up to **19,000 other ImageNet images**. This oversampling strategy is deliberate: we want the SAE to develop dedicated latents for zebra-related visual features (stripes, texture, shape) so that they can be cleanly identified and ablated during the unlearning step.

Why Zebras?

Zebras serve as a prototypical 'concept to unlearn' in our experiments. Their distinctive stripe pattern makes them easy to identify in activation space. Once we confirm that SAE latents correspond to zebra features (via feature visualization or probing), we can zero out those latents during inference — causing the VLM to effectively 'forget' what zebras look like, without affecting recognition of horses or other animals.

5. How SAE-Based Vision Unlearning Works

5.1 The Unlearning Pipeline

After training, the SAE is used as an interpretability and intervention tool during inference:

- **Step 1 — Identify target latents:** Run the trained SAE on images of the concept to unlearn (e.g., zebra). Record which latents activate most strongly and consistently — these are the 'zebra features'.
- **Step 2 — Confirm feature semantics:** Optionally validate via feature visualisation (upsample activations to image patches) or probing (train a linear classifier on the latent to predict class).
- **Step 3 — Patch inference:** At inference time, run images through CLIP up to layer 22, pass activations through the SAE encoder, zero out target latents, decode back, and continue through the rest of the VLM.
- **Step 4 — Verify unlearning:** Measure that the VLM's performance on zebra-related tasks has degraded, while performance on unrelated tasks (e.g., identifying horses or cats) remains intact.

5.2 Why This Is Effective

- **Surgical:** Only the relevant latents are ablated. The rest of the 4,096-dimensional sparse representation is untouched.
- **Reversible:** No weights are modified. Unlearning can be 'turned off' by not applying the ablation.
- **Generalises:** Because SAE latents are learned from a diverse dataset, they capture abstract visual features (not just memorised images).
- **Transparent:** The SAE makes the internal visual representation human-readable — we can inspect which patches activate which latents, providing an audit trail for the unlearning decision.

6. Metrics Tracked During Training

Metric	Meaning	Good Sign
MSE Loss	Mean squared error between input and reconstructed activation vector.	Decreasing; < 0.1 for well-trained SAEs
L1 Loss	Mean absolute activation magnitude — proxy for sparsity.	Stable; driven by λ coefficient

L0 (active features)	Mean number of latents with activation > 0 per token.	Low; ideally 20–80 for 4× SAE
Explained Variance	1 − Var(x−x̂)/Var(x). Captures % of variance reconstructed.	Close to 1.0 (>0.9 is good)
Dead Features (%)	% of latents never active in past 50K tokens.	< 5%; high % = wasted capacity

7. Output Artifacts

File	Purpose
sae_layer{L}_final.pt	SAE weights only — for inference and unlearning application
sae_layer{L}_latest.pt	Full checkpoint (weights + optimizer + early stopper state) — for resuming training
sae_layer{L}_metrics.json	Step-by-step log of MSE, L1, L0, EV, dead features
activations_v3/layer_{L}/chunk_NNNN.pt	Raw frozen CLIP activations — reused across training runs
activations_v3/layer_{L}/stats.pt	Per-layer mean and mean-norm for normalisation
activations_v3/extraction_summary.json	Metadata about the extraction run (image count, layers, models)

8. Summary & Next Steps

8.1 Where We Are

- v2 (TopK) established a working multi-layer pipeline with hard-sparsity SAEs trained on CLIP ViT-L activations at layers 12, 17, and 22.
- v3 (L1-ReLU) refines the approach: smaller (4×), faster, research-aligned, with early stopping, LR formula from Pach et al. 2025, and skip-if-exists caching.
- Both versions include zebra oversampling to ensure the SAE develops concept-specific latents suitable for unlearning experiments.

8.2 Next Steps

- **Feature attribution:** Build visualisation tools that map high-activating SAE latents back to image patches — confirming which latents encode 'zebra-ness'.
- **Unlearning experiment:** Apply targeted latent ablation at inference time and benchmark VLM performance on zebra vs. non-zebra tasks.
- **Multi-concept unlearning:** Extend beyond zebra to other concepts; test for cross-concept interference.
- **Layer ablation study:** Compare unlearning effectiveness when SAE is applied at layer 12 vs. 17 vs. 22.
- **Evaluate sparsity trade-offs:** Sweep λ values (L1 coefficient) to find the optimal sparsity vs. reconstruction fidelity trade-off for this use case.

End of Document